IS1204 Fundamentals of Software Engineering

H N D THILINI

Architectural Design

Topics Covered

- Architectural design
- Architectural design decisions
- Architectural views
- Architectural styles/patterns

What is an Architecture

Learning objectives

- Understand why the architectural design of software is important.
- Understand the decisions that have to be made about the system architecture during the architectural design process.
- Understand the common architectural styles/patterns that are often used to organize the system architecture.

An Architecture is

"the fundamental organization of a system embodied in its **components**, their **relationships** to each other, and to the **environment**, and the **principles** guiding its design and evolution."

(IEEE)

- **Every system has an architecture, whether or not it is documented and understood.**

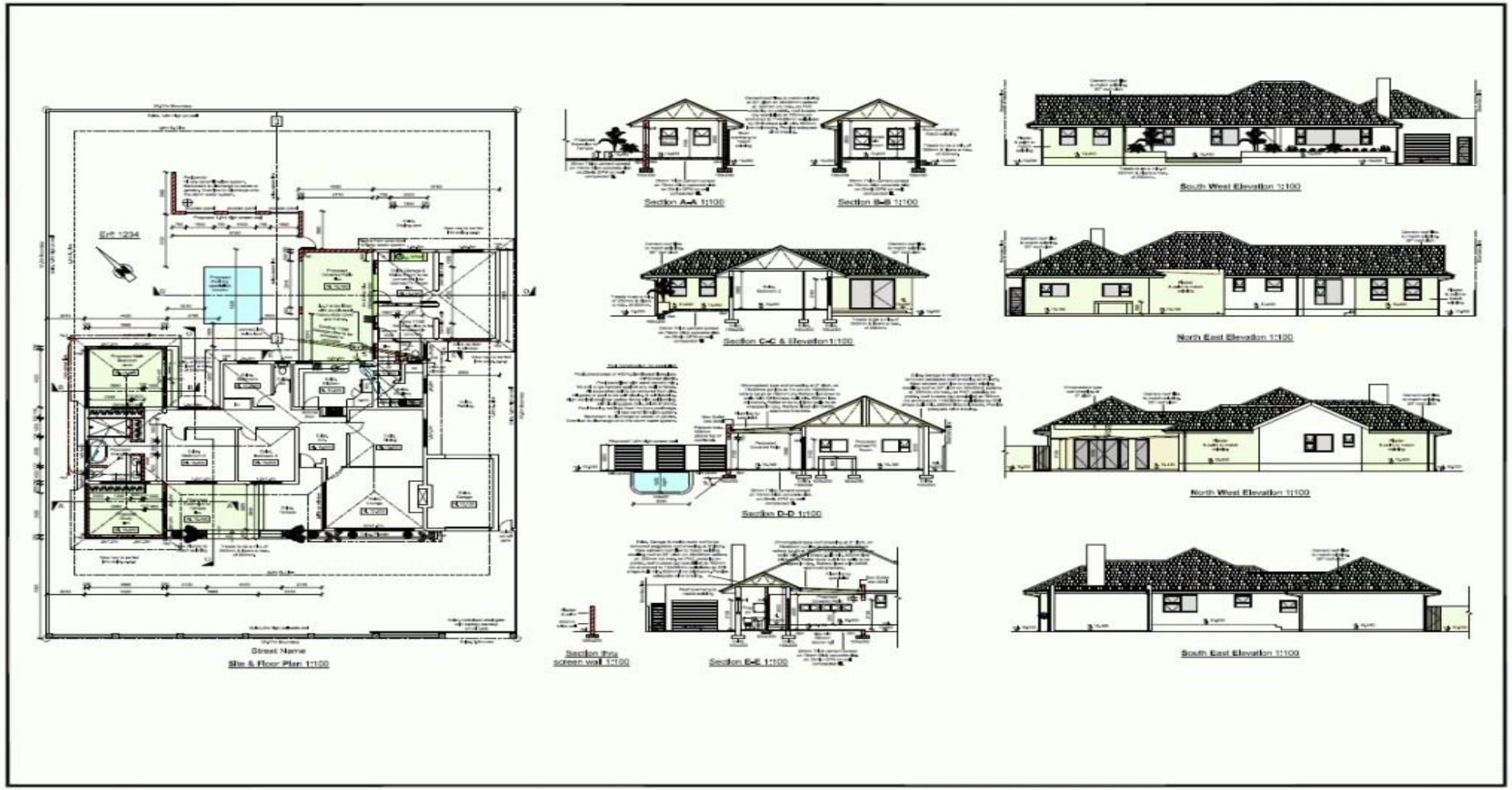
Why do we need an Architecture?

- Understand the system
  - Complexity
- Organize the development
  - Architectural partitioning
- Reuse
- Evolution
  - Changes and dependencies

What is a Software Architecture

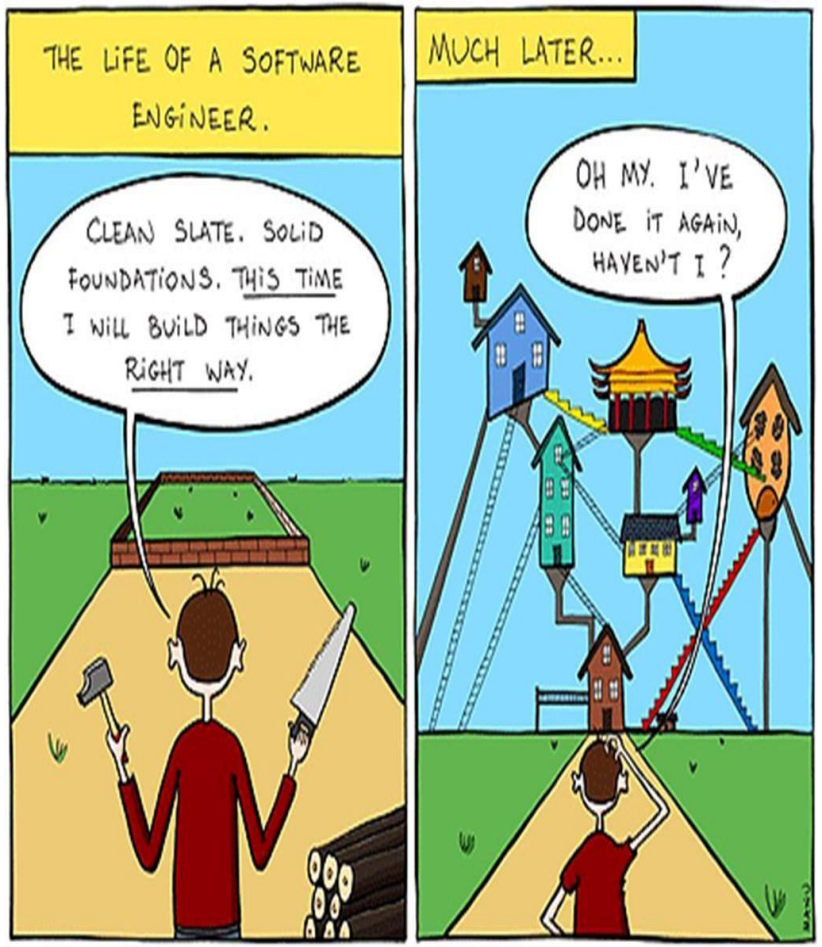
- How a software system should be organized?
- Identifies the main structural components in a system and its relationships.
- Overall system design
  - Subsystems
  - Interconnections
  - Collaborations
- Architectural model -The output of the architectural design process
  - how the system is organized as a set of communicating components.

Architectural Model



Agility and System Architecture

- Overall systems architecture design should be done at an early stage of agile processes.
- Incremental development of architecture is NOT successful.
- Refactoring the system architecture is expensive
  - Can affect so many components in the system



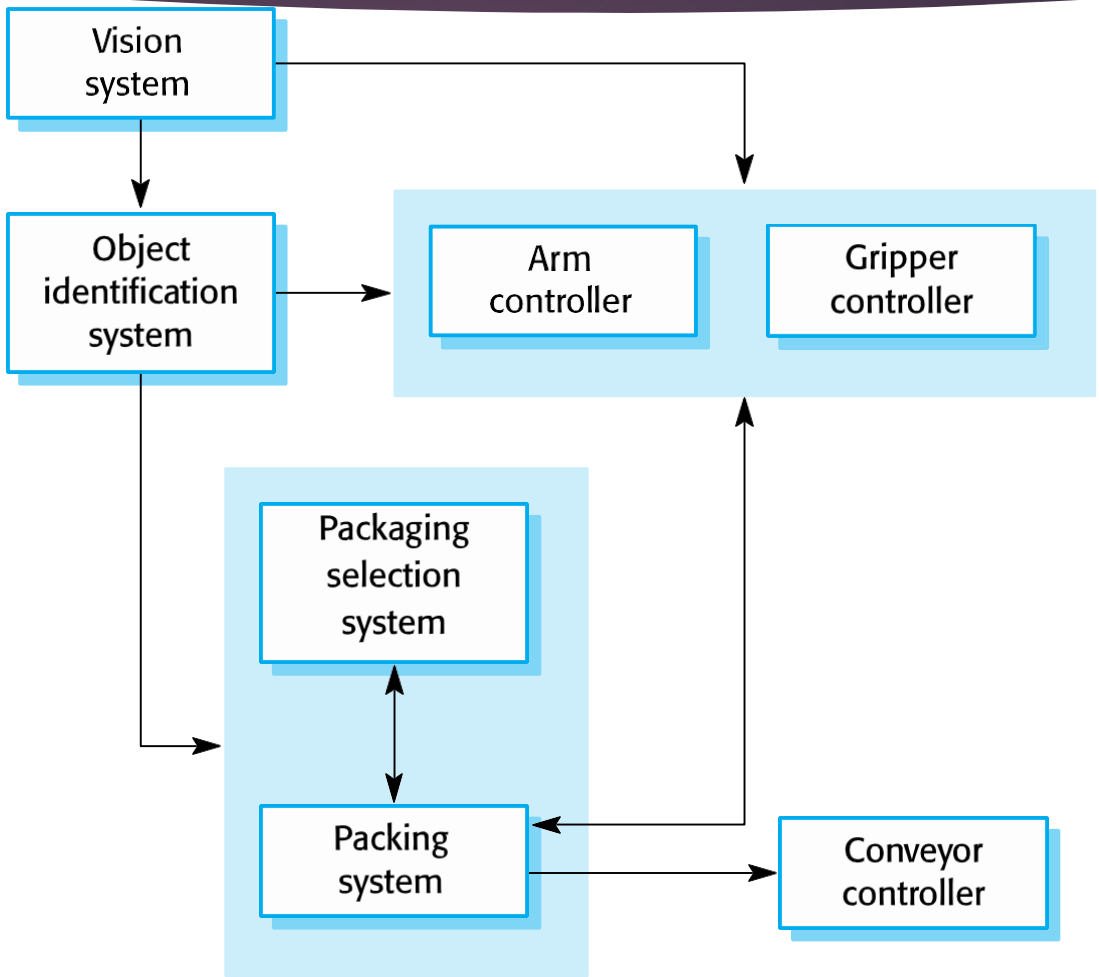
Advantages of explicitly designing and documenting software architecture

- Stakeholder communication
  - High level presentation of the system may be used as a focus of discussion by system stakeholders.
- System analysis
  - Means that analysis of whether the system can meet its non- functional requirements is possible.
- Large-scale reuse
  - The architecture may be reusable across a range of systems
  - System architecture is often the same for similar requirements

Basic principles considered by architects

- Abstraction, Encapsulation
  - What it does from How it does
  - Hide the internal structure
- Separate of concerns
  - Interface / implementation
  - Data / transactions
  - Policy / business logic
- Modularity / Structuring / Organization
  - Modular decomposition
  - Coupling and cohesion

The architecture of a packing robot control system

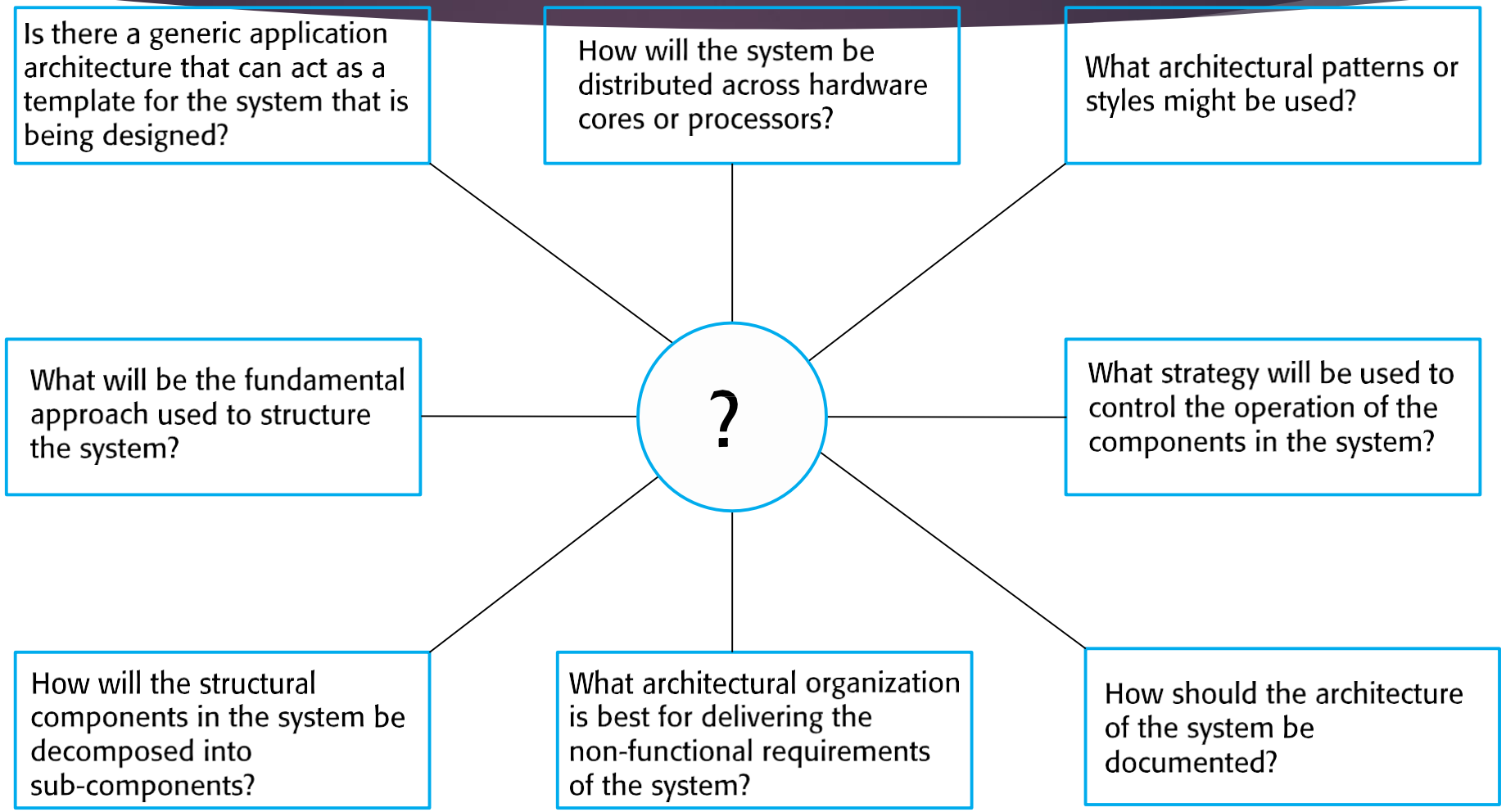


ARCHITECTURAL DESIGN DECISIONS

Architectural Design Decisions

- A creative process.
- The activities within the process differs depending on
  - the type of system being developed,
  - the experience of the architect,
  - system requirements.
- Need to thought of structural decisions.
  - Affect the system and its development process

Architectural Design Decisions



Architectural Design Decisions

Is there
architec
templat
being d

Is there a generic application architecture that can act as a template for the system that is being designed?

What architectural patterns or styles might be used?

What w
approa
the system?

What strategy will be used to control the operation of the components in the system?

How will the structural components in the system be decomposed into sub-components?

What architectural organization is best for delivering the non-functional requirements of the system?

How should the architecture of the system be documented?

Architectural Design Decisions

Is there a generic applicat
architecture that can act
template for the system
being designed?

ctural patterns or
be used?

How will the system be distributed across hardware cores or processors?

What will be the fundan
approach used to struct
the system?

gy will be used to
operation of the
components in the system?

How will the structural components in the system be decomposed into sub-components?

What architectural organization is best for delivering the non-functional requirements of the system?

How should the architecture of the system be documented?

Architectural Design Decisions

Is there a generic application architecture that can act as a template for the system that is being designed?

How will the system be distri cores

What architectural patterns or styles might be used?

What will be the fundamental approach used to structure the system?

!

control the operation of the components in the system?

How will the structural components in the system be decomposed into sub-components?

What architectural organization is best for delivering the non-functional requirements of the system?

How should the architecture of the system be documented?

Architectural Design Decisions

Is there a generic application architecture that can act as a template for the system that is being designed?

How will the system be distributed across hardware cores or processors?

What architectural patterns or styles might be used?

What will be the fundamental approach used to structure the system?

What strategy will be used to control the operation of the components in the system?

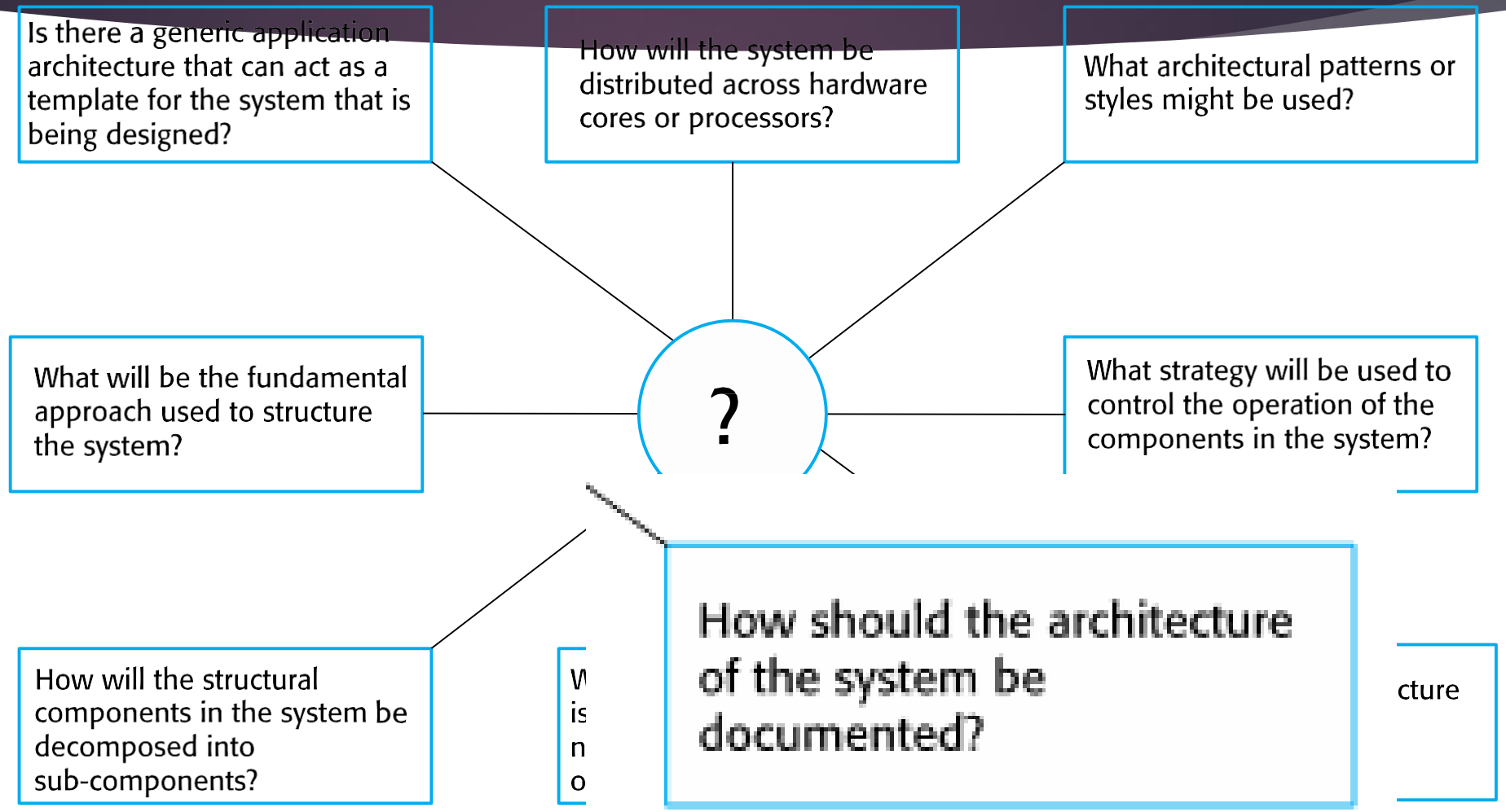
ed to
the
m?

How will the structural components in the system be decomposed into sub-components?

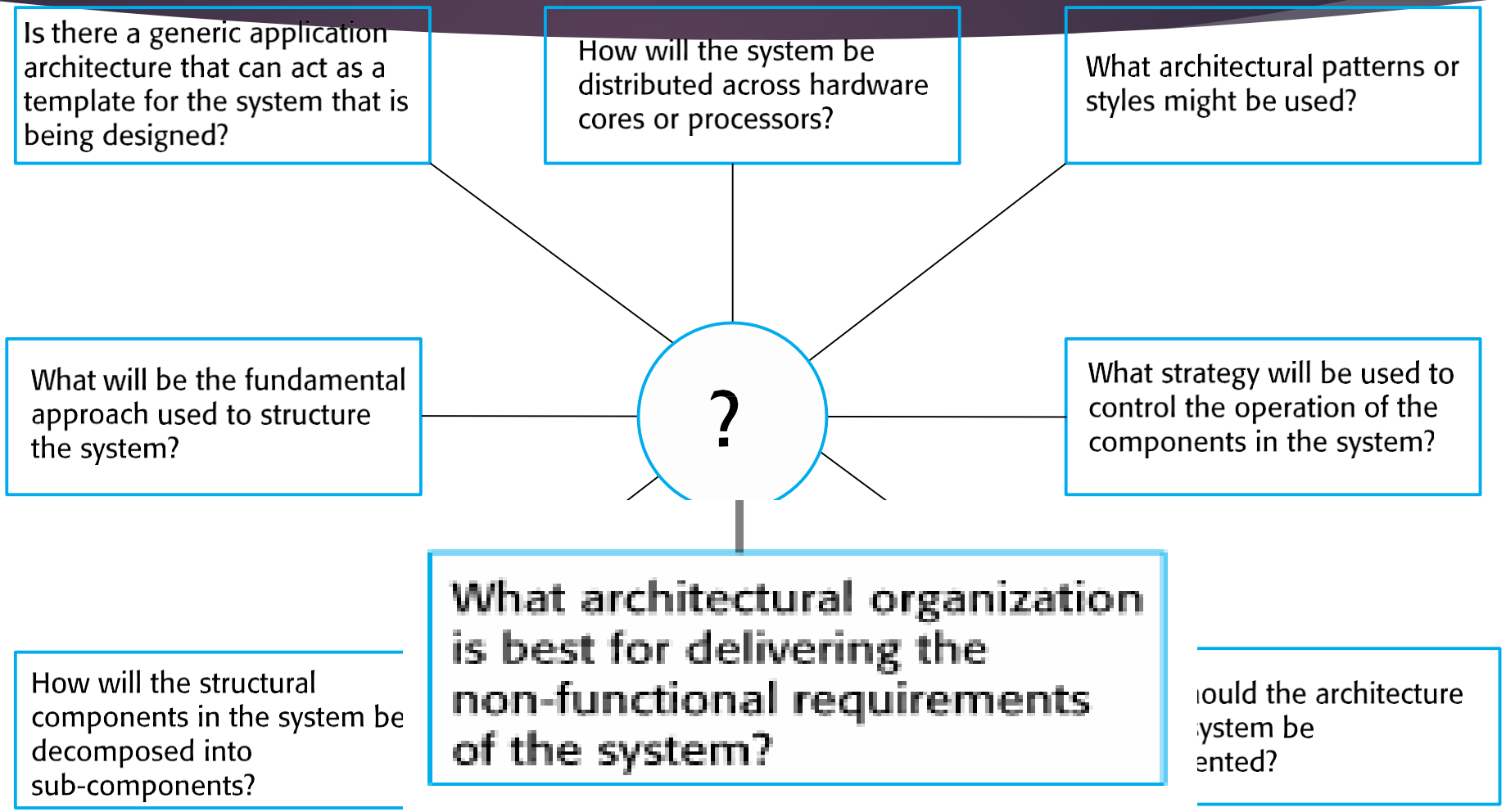
What architectural organization is best for delivering the non-functional requirements of the system?

How should the architecture of the system be documented?

Architectural Design Decisions



Architectural Design Decisions



Choice of Architectural Style and Non functional

- Performance
  - Increase the response time. Localize critical operations and minimize communications. Use large rather than fine-grain components.
- Security
  - Use a layered architecture with critical assets in the inner layers.
- Safety
  - Localize safety-critical features in a small number of sub- systems.
- Availability
  - Include redundant components and mechanisms for fault tolerance.
- Maintainability
  - Use fine-grain, replaceable components

Architectural Design Decisions

Is there a generic application architecture that can act as a template for the system that is being designed?

How will the system be distributed across hardware cores or processors?

What architectural patterns or styles might be used?

What will be the fundamental approach used to structure the system?

?

What strategy will be used to control the operation of the components in the system?

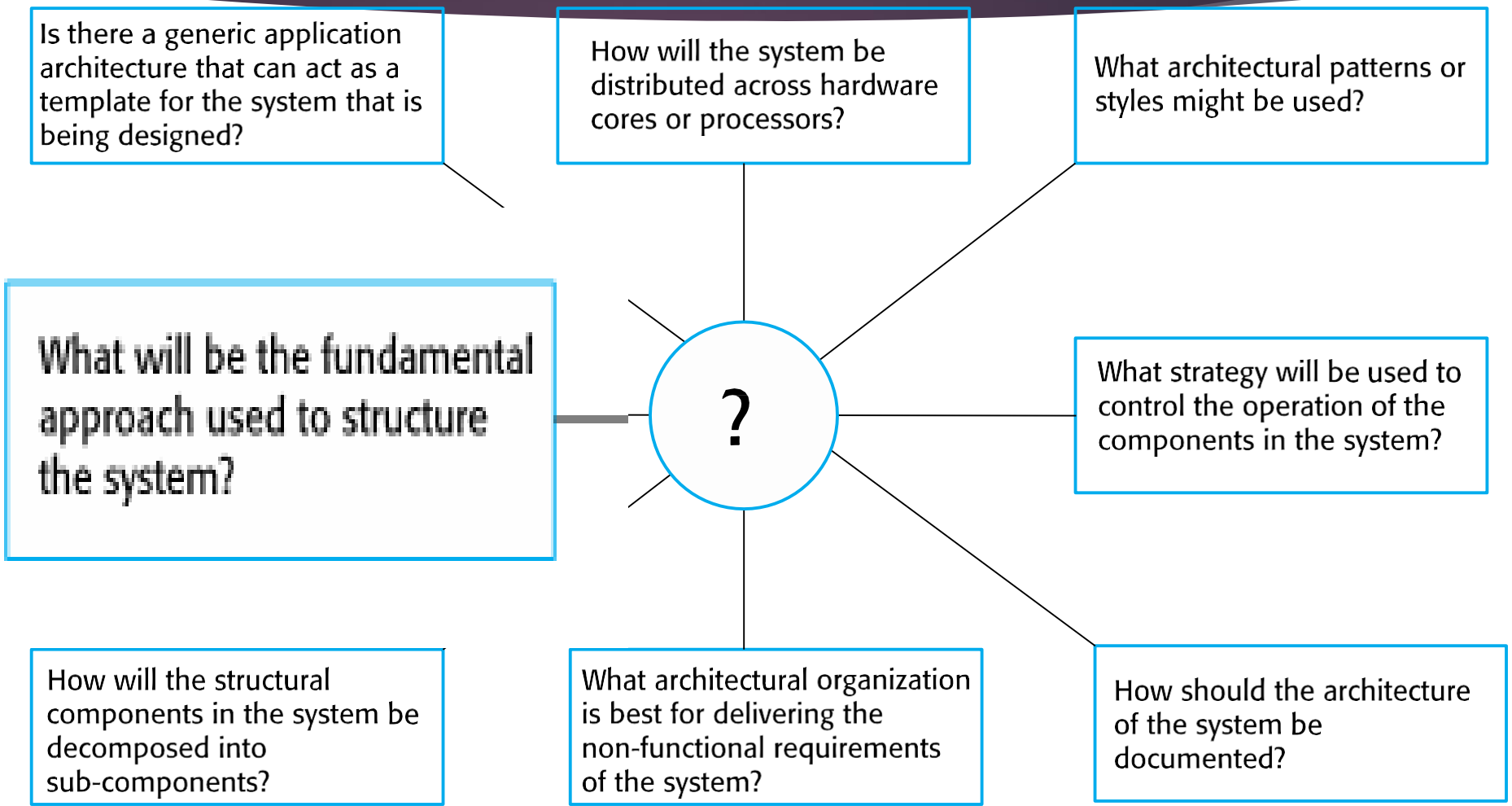
How will the structural components in the system be decomposed into sub-components?

ization

ents

How should the architecture of the system be documented?

Architectural Design Decisions



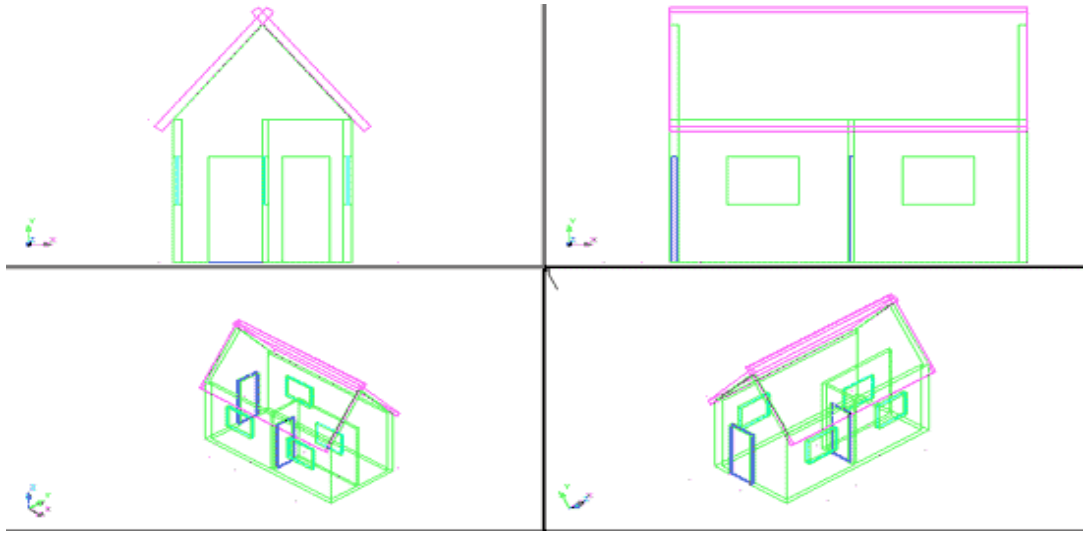
ARCHITECTURAL VIEWS

Architectural views

- What views or perspectives are useful when designing and documenting a system's architecture?
- What notations should be used for describing architectural models?
- Each architectural model only shows one view or perspective of the system.
  - It might show how a system is decomposed into modules, how the run-time processes interact or the different ways in which system components are distributed across a network. For both design and documentation, you usually need to present multiple views of the software architecture.

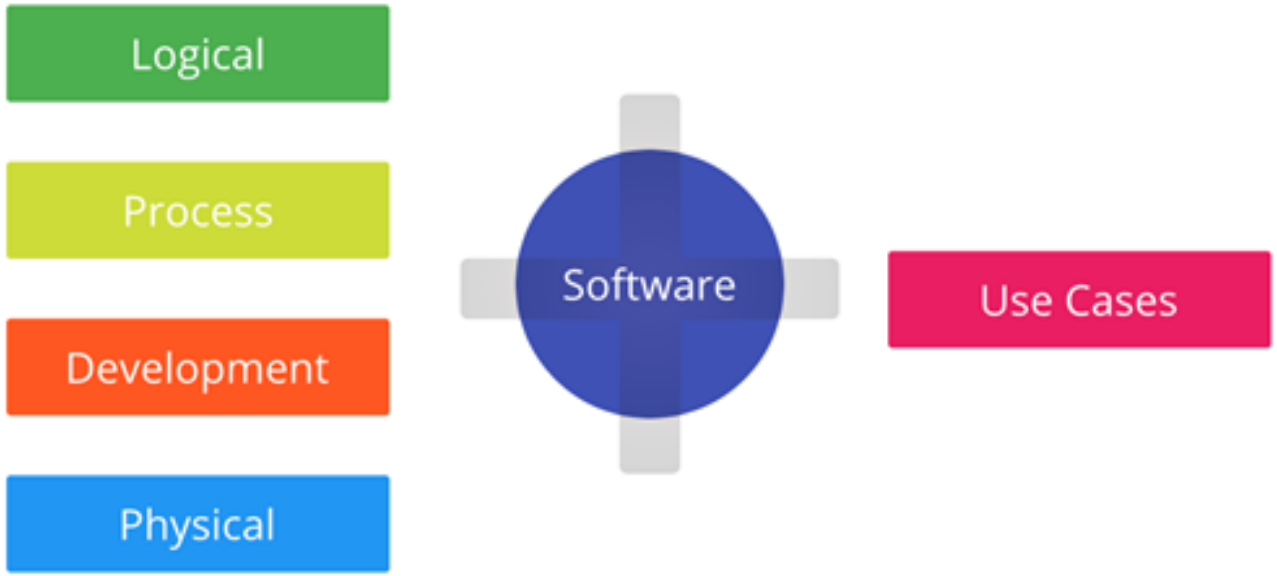
What is a View Model?

- A framework
- Defines set of views to be used in designing software architecture.
- View is a representation of the whole system from different perspectives.



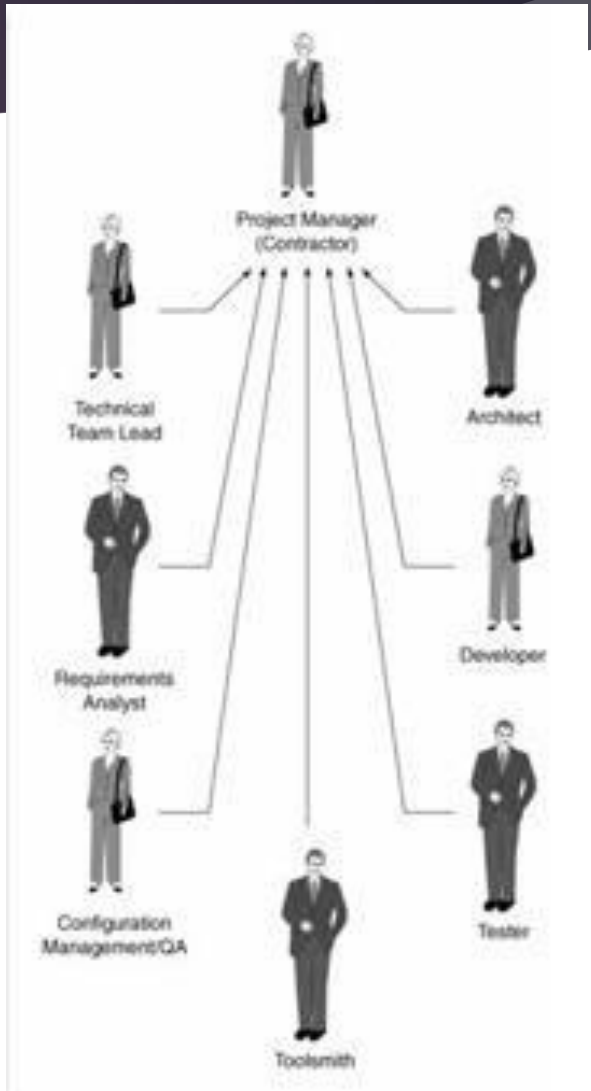
4 + 1 VIEW MODEL OF SOFTWARE ARCHITECTURE

Designed by : Philippe Kruchten

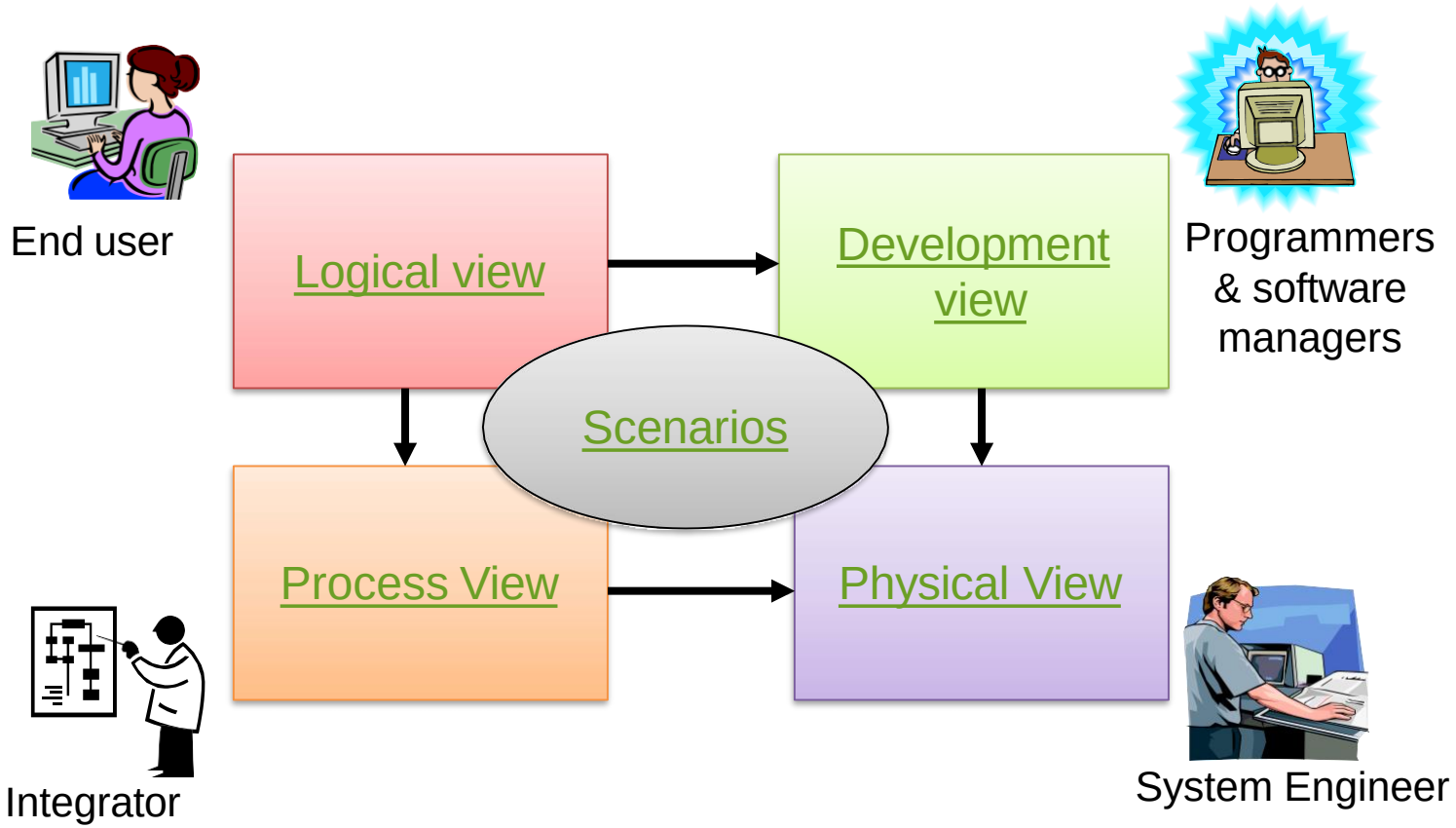


Intent of 4 + 1 View Model

- To separate different aspects of a software system into different views of the system.
- Why?
  - Different stakeholders have different interest of the system



4+1 View Model of Architecture



4 + 1 View Model Architecture

- Explains the system from the viewpoint of different stakeholders.
- 4 – Views
  - A logical view - shows the key abstractions in the system as objects or object classes.
  - A process view - shows how, at run-time, the system is composed of interacting processes.
  - A development view - shows how the software is decomposed for development.
  - A physical view - shows the system hardware and how software components are distributed across the processors in the system.
- Plus one view to represent use cases / scenarios

Logical View

Viewer: End-user

Considers: Functional requirements - What the system should provide in terms of services to its users.

- Shows the components of the system and their interaction.

Process View

(The process decomposition)

Viewer: Integrators

Considers: Non - functional requirements (concurrency, performance, scalability)

- Processors / workflow of a system and how processors communicate with each other.

Development View

(Subsystem decomposition)

Viewer: Programmers and Software Managers

Considers: Organization(Hierarchy of layers, software management, reuse, constraints of tools)

- Represents the building block view of the system

Physical View

(Mapping the software to the Hardware)

Viewer: System Engineers

Considers: system hardware and how software components are distributed across processors in the system. Non-functional req. regarding underlying hardware (Topology, Communication)

- Represents the building block view of the system

Scenarios / Use Case View

(Putting it all together))

Viewer: All users of other views and Evaluators

Considers: System consistency, validity

- Validation and illustration to show the design is complete.

ARCHITECTURAL PATTERNS / STYLES

Architectural Patterns

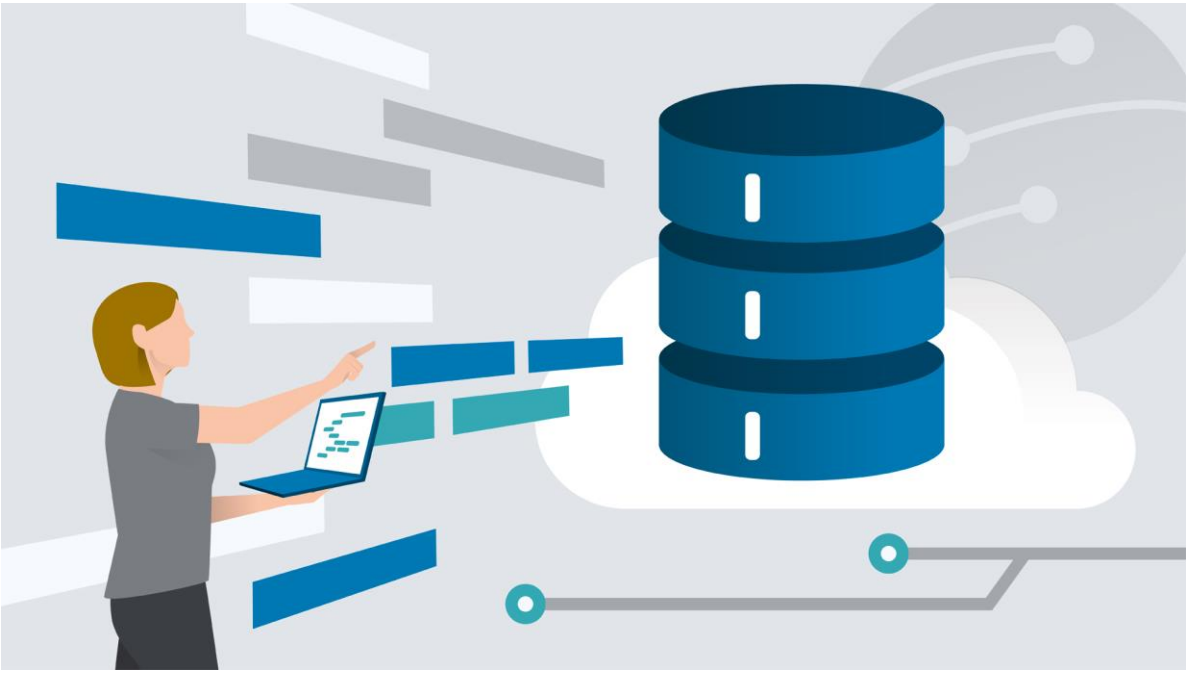
- A general, reusable solution to a commonly occurring problem in software architecture within a given context
- Patterns are a means of representing, sharing and reusing knowledge.
- Patterns should include information about when and where it is applicable or not.
- Patterns are described in a standard way using a mixture of narrative description / diagrams.

Architectural Paterns

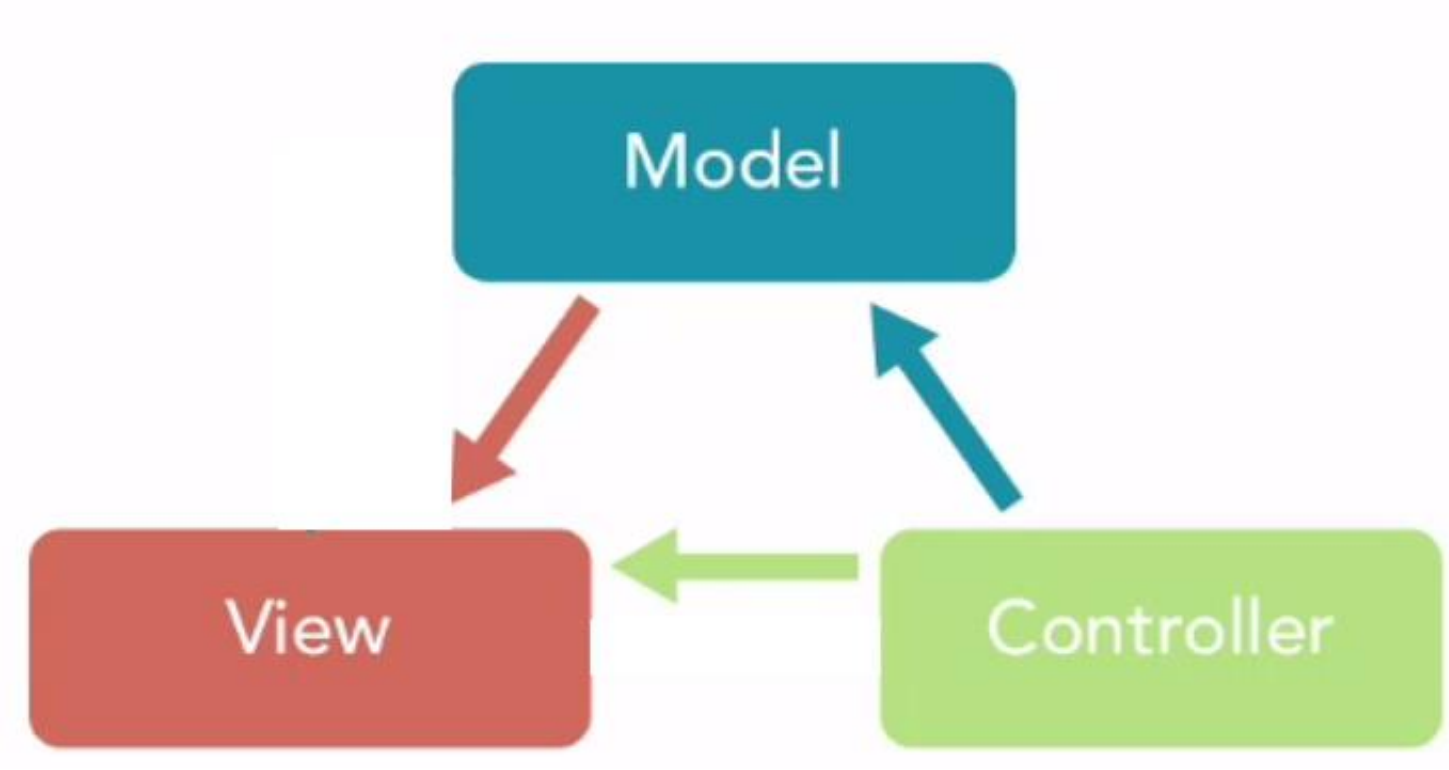
1. Model-View-Controller (MVC) architecture
2. Layered architecture
3. Repository architecture
4. Client-Server architecture
5. Pipe & Fiter architecture

Architectural Patterns/styles Examples

MVC



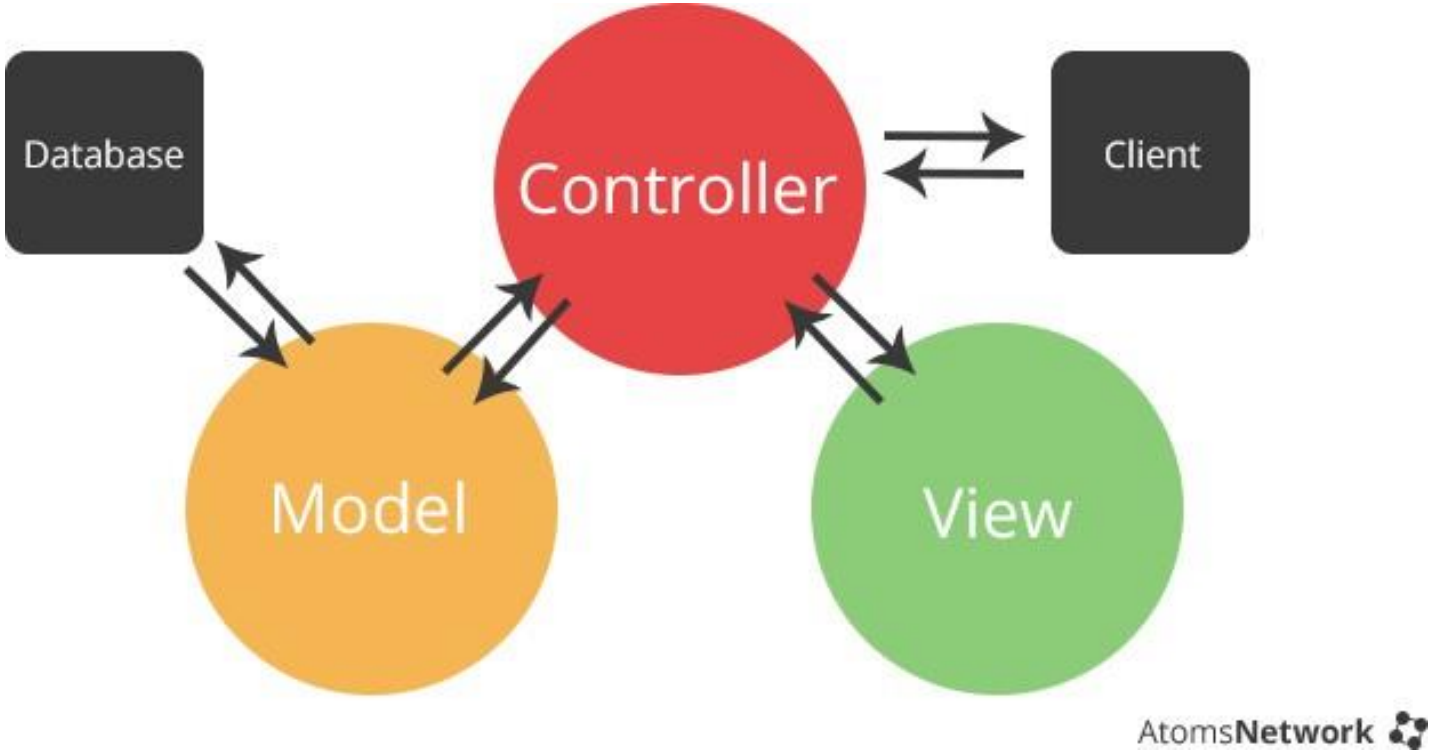
What is MVC



MVC Architecture

- **Model** - Manages the system data and business logic. Maintains data and associated operations on that data.
- **View** - Defines and manages how the data is presented to the user.
- **Controller** - Manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.

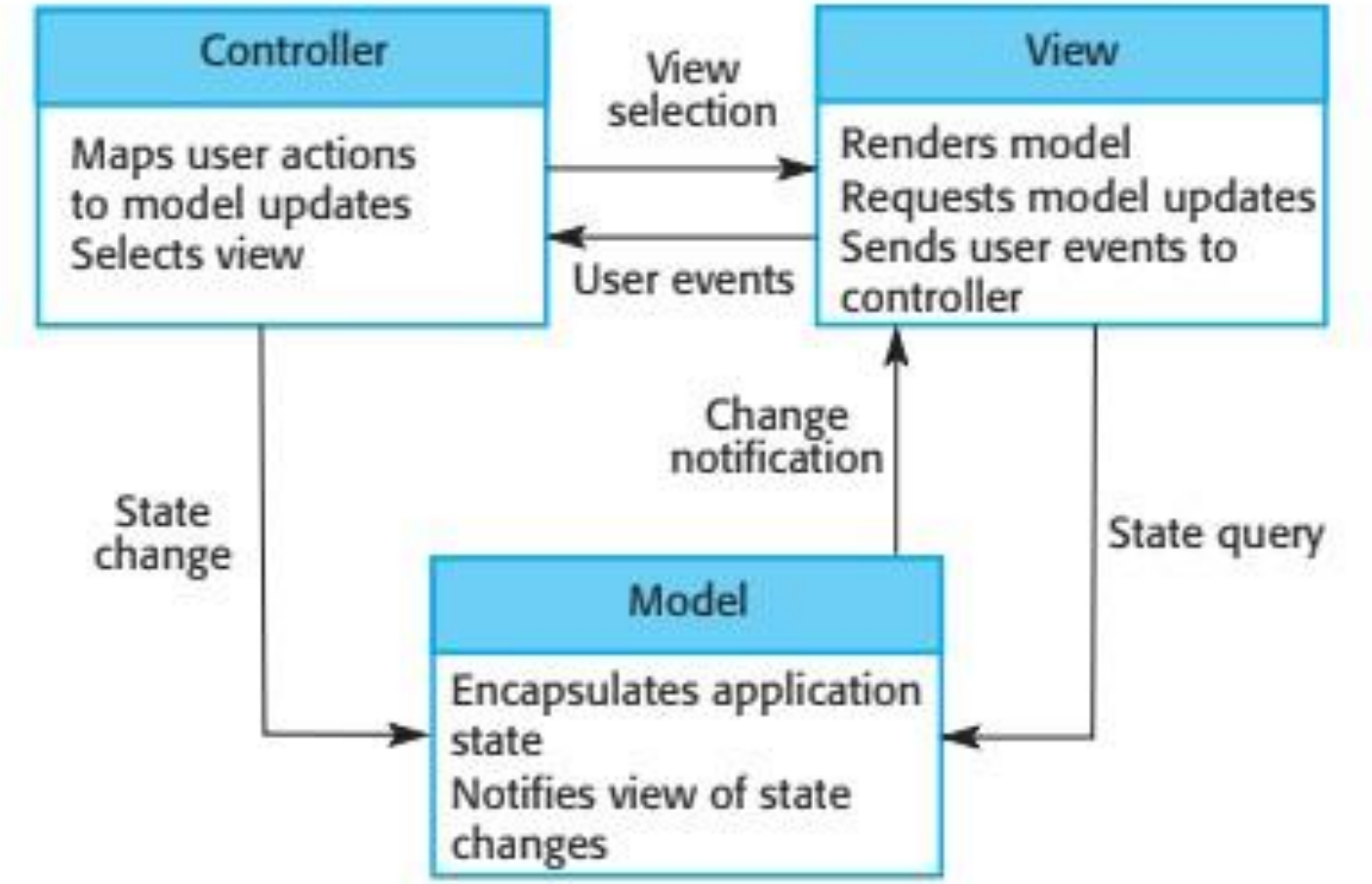
The Organization of MVC



MVC Narrative Description

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.5.
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways, with changes made in one representation shown in all of them.
Disadvantages	May involve additional code and code complexity when the data model and interactions are simple.

MVC Graphical Representation



MVC

Points to Remember :

1) MVC stands for Model, View and Controller.

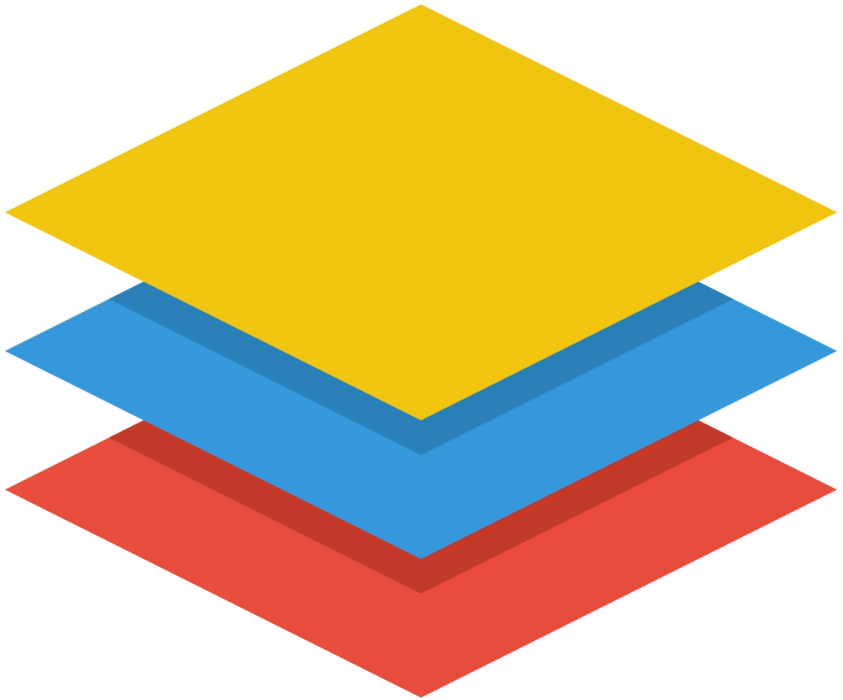
2) Model is responsible for maintaining application data and business logic.

3) View is a user interface of the application, which displays the data.

4) Controller handles user's requests and renders appropriate View with Model data.

Architectural Patterns/styles Examples

LAYERED ARCHITECTURE



Layered Architecture

- Organizes the system into layers with related functionality associated with each layer.
- A layer provides services to the layer above it
- The lowest-level layers represent core services that are likely to be used throughout the system.
- Supports the incremental development of sub-systems in different layers.
- When a layer interface changes, only the adjacent layer is affected.

The Layered Architecture

Name	Layered architecture
Description	Organizes the system into layers, with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system. See Figure 6.8.
Example	A layered model of a digital learning system to support learning of all subjects in schools (Figure 6.9).
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multilevel security.
Advantages	Allows replacement of entire layers as long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

A Generic Layered Architecture

User interface

User interface management
Authentication and authorization

Core business logic/application functionality
System utilities

System support (OS, database etc.)

Architectural Patterns/styles Examples

REPOSITORY ARCHITECTURE

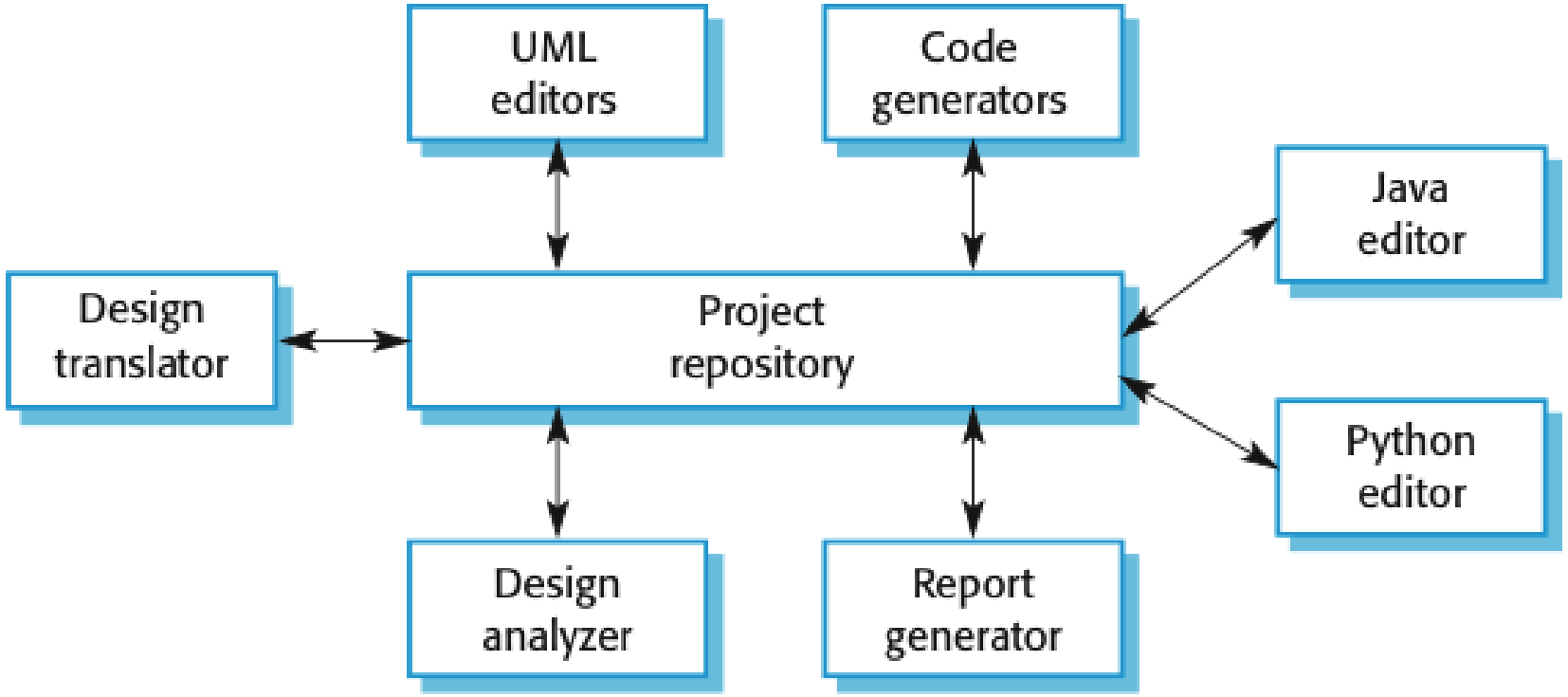
Repository Architecture

- Describes how a set of interacting components can share data.
- Sub-systems must exchange data. This may be done in two ways
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used
- Concerned with the static structure, not the run time.

The Repository Architecture

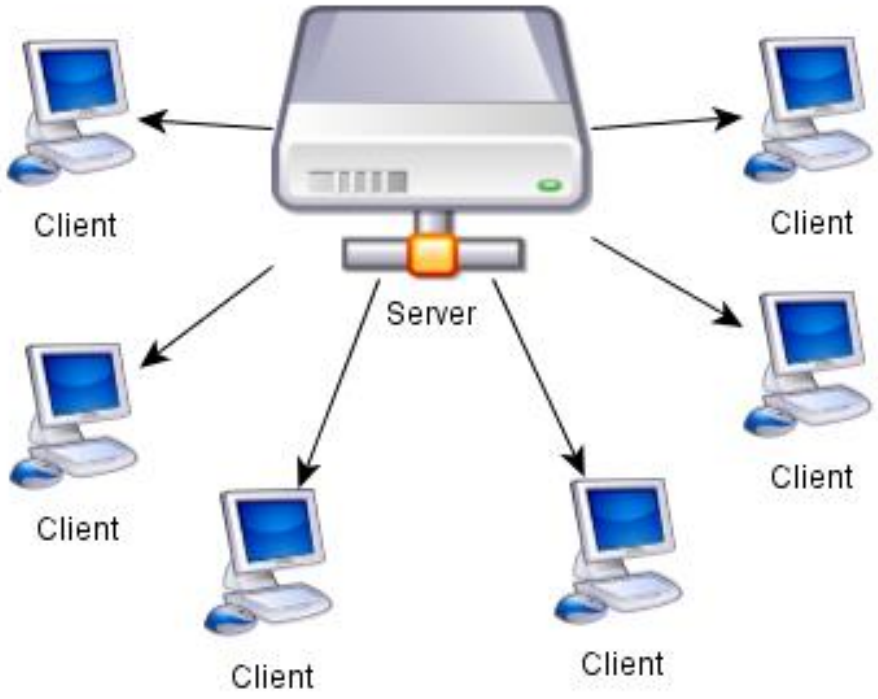
Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.11 is an example of an IDE where the components use a repository of system design information. Each software tool generates information, which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

The Repository Architecture



Architectural Patterns/styles Examples

CLIENT-SERVER ARCHITECTURE



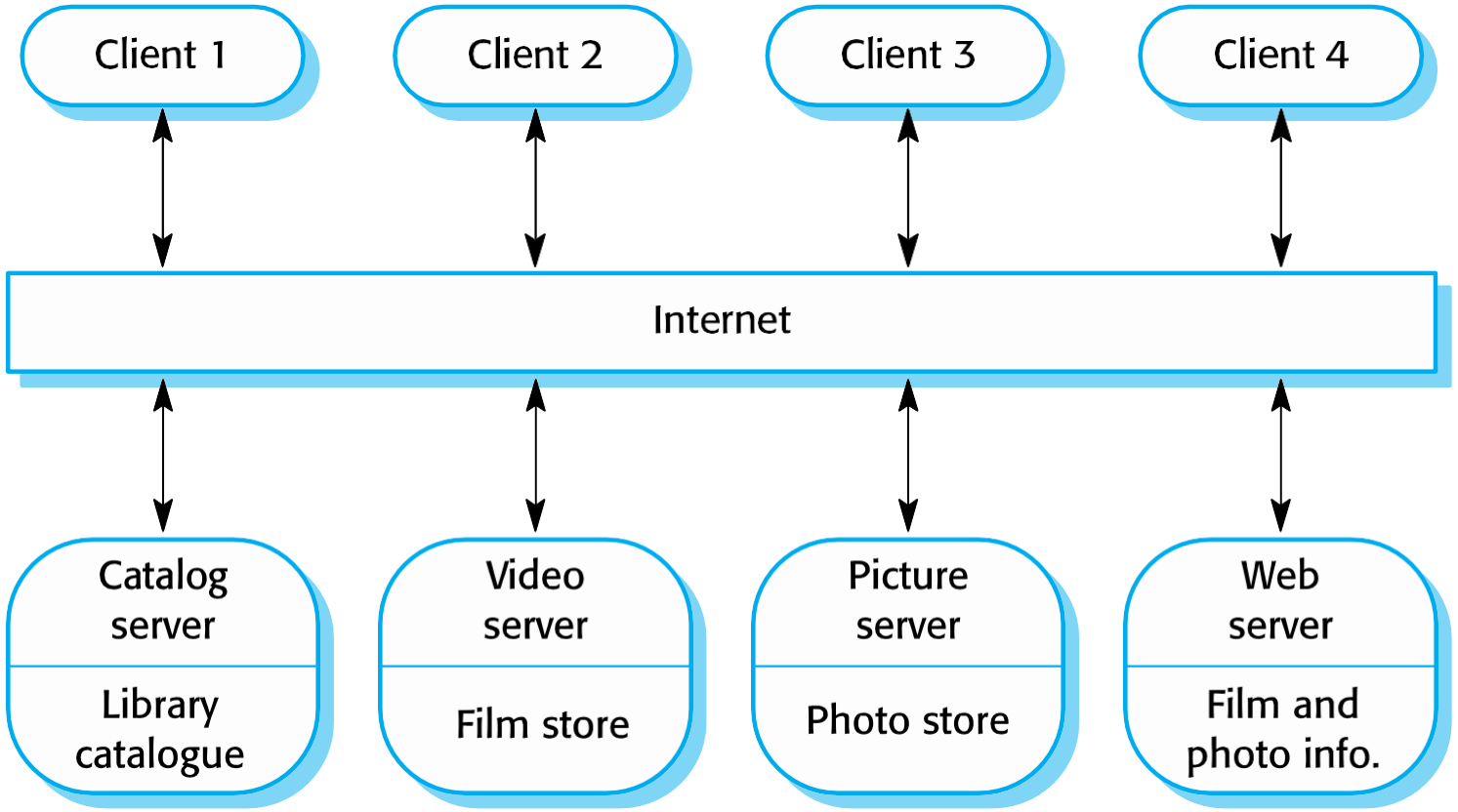
Client-Server Architecture

- Distributed system model which shows how data and processing is distributed across a range of components.
  - Can be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

The Client-Server Architecture

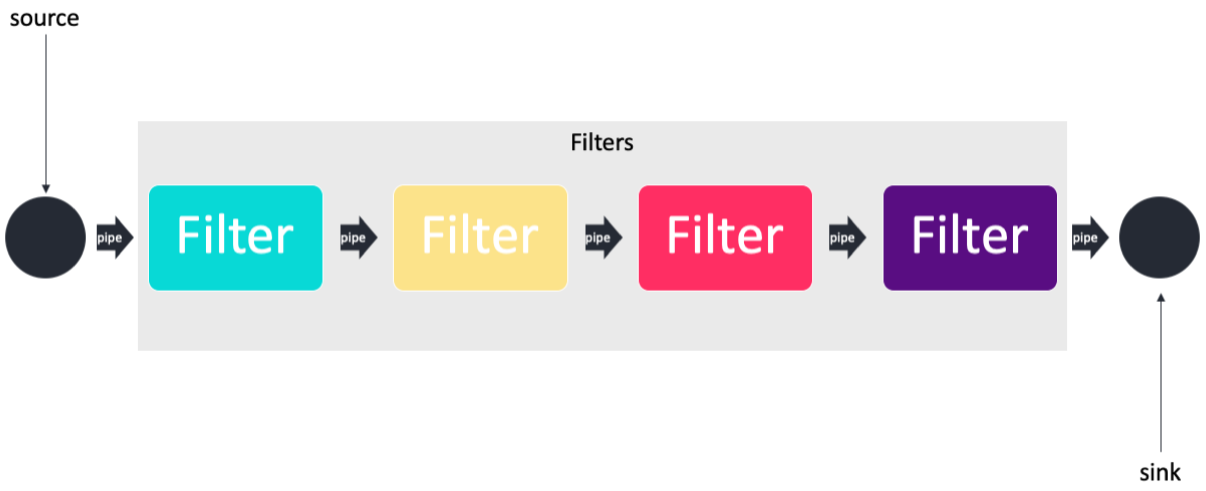
Name	Client–server
Description	In a client–server architecture, the system is presented as a set of services, with each service delivered by a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.13 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure and so is susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. Management problems may arise if servers are owned by different organizations.

A Client-Server architecture for a film library



Architectural Patterns/styles Examples

PIPE AND FILTER ARCHITECTURE



Pipe and Filter Architecture

- Model of the run time organization of the system.
- Functional transformations process their inputs and produce outputs.
- Data flows from one to another and transforms as it moves through the sequence.
- Each processing step is implemented as a transform.
- Input data flows through these transforms until converted to outputs.
- Transformations may execute sequentially / parallel.

Pipe and Filter Architecture

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data-processing applications (both batch and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

Summary

- A software architecture is a description of how a software system is organized.
  - Properties such as performance, security, and availability are influenced by the architecture used.
- Architectural design decisions include
  - decisions on the type of application, the distribution of the system, the architectural styles to be used, and the ways in which the architecture should be documented and evaluated.
- Architectures may be documented from several different perspectives or views.
  - Possible views include a conceptual view, a logical view, a process view, a development view, and a physical view.

Summary

- Architectural patterns, a means of reusing knowledge about generic system architectures.
  - Describe the architecture, explain when it may be used, and point out its advantages and disadvantages.
- Commonly used Architectural patterns include
  - model-view-controller,
  - layered architecture,
  - repository,
  - client–server, and
  - pipe and filter.

Group Assignment 05 – Analyzing Software Architectural Patterns

- Task:
  - Prepare a presentation analyzing the different software architectural patterns. Your presentation should include all the architectural patterns discussed in the class and at least three (3) additional architectural patterns that are commonly used in software development.
  - For each pattern:
    - give a brief description of the pattern
    - explain at least one use case of the pattern
    - Discuss the limitations of the pattern.